

they're operating on huge functions backed by millions of lines of code or functions that are simple one-liners. Polymorphic, higher-order functions often end up being extremely widely applicable, precisely because they say nothing about any particular domain and are simply abstracting over a common pattern that occurs in many contexts. For this reason, programming in the large has much the same flavor as programming in the small. We'll write a lot of widely applicable functions over the course of this book, and the exercises in this chapter are a taste of the style of reasoning you'll employ when writing such functions.

2.7 Summary

In this chapter, we learned enough of the Scala language to get going, and some preliminary functional programming concepts. We learned how to define simple functions and programs, including how we can express loops using recursion; then we introduced the idea of higher-order functions, and we got some practice writing polymorphic functions in Scala. We saw how the implementations of polymorphic functions are often significantly constrained, such that we can often simply “follow the types” to the correct implementation. This is something we'll see a lot of in the chapters ahead.

Although we haven't yet written any large or complex programs, the principles we've discussed here are scalable and apply equally well to programming in the large as they do to programming in the small.

Next we'll look at using pure functions to manipulate data.